

Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3067 Redes
Sección 20

Proyecto 1 - Finance MCP

Uso de un protocolo existente

Diego López
Carné 23747
1 de septiembre de 2026

Índice

Resumen ejecutivo	2
Objetivos y alcance	2
Arquitectura del sistema	3
Implementación manual de MCP y JSON-RPC	5
Especificación de Finance MCP	7
Transportes, endpoints y configuración	8
Ejemplos de uso	8
Interfaz Web y controles HCI	9
Verificación y calidad	9
Clasificación JSON-RPC y MCP	10
Análisis por capas	10
Enlace	10
Red	10
Transporte y TLS	10
Aplicación	11
Dificultades, soluciones y aprendizajes	12
Conclusiones y comentario del proyecto	12
Referencias	12

Resumen ejecutivo

El proyecto implementa un sistema de administración financiera para una pequeña empresa ficticia mediante Model Context Protocol, MCP. La solución integra una interfaz Web, un Host conversacional, un cliente DeepSeek y tres servidores MCP. Finance MCP provee el dominio financiero. Filesystem MCP y Git MCP proveen capacidades locales restringidas. El servidor Finance MCP fue implementado manualmente con JSON-RPC 2.0 y MCP 2025-11-25, sin un SDK MCP.

Finance MCP ofrece veinticuatro tools. Nueve son de lectura y quince son de escritura. Los cálculos de saldos, flujo de caja, proyecciones e inventario permanecen dentro del servidor financiero. El Host descubre las tools, conserva las sesiones, exige confirmación explícita antes de toda escritura y mantiene logs MCP sanitizados. Finance MCP puede operar por STDIO en modo local o por Streamable HTTP en modo remoto. La interfaz Web mantiene separadas las capas de presentación, Host, MCP y persistencia.

La validación incluyó pruebas unitarias, integraciones con PostgreSQL efímero, transporte HTTP, un escenario local de extremo a extremo y una validación remota controlada. La evidencia de red se obtuvo con una sesión Host a Finance MCP remoto. La captura confirmó lifecycle, discovery, una lectura y cierre de sesión sobre HTTP/1.1 protegido por TLS 1.3.

Objetivos y alcance

El objetivo general consiste en implementar y validar un Host que coordine servidores MCP locales y remotos mediante un protocolo estándar, con un caso de uso financiero y una interfaz Web.

Los objetivos específicos incluyen implementar JSON-RPC y el lifecycle MCP de forma manual, definir una especificación pública de Finance MCP, integrar servidores oficiales de Filesystem y Git, conservar contexto conversacional por pestaña, registrar interacciones MCP sanitizadas y demostrar la comunicación remota mediante una captura de red.

El alcance corresponde a un MVP. La interfaz ofrece un dashboard de solo lectura, chat, logs MCP y confirmaciones explícitas. Finance MCP funciona localmente mediante STDIO y remotamente mediante Streamable HTTP. Filesystem MCP y Git MCP permanecen locales y restringidos a directorios aislados.

Quedan fuera del alcance autenticación, alta disponibilidad, sincronización entre bases locales y remotas, reintentos automáticos, fallback de transporte, SSE, exportación financiera en PDF y operaciones Git remotas. Estas exclusiones reducen superficie de riesgo y mantienen el proyecto centrado en la interoperabilidad MCP.

Arquitectura del sistema

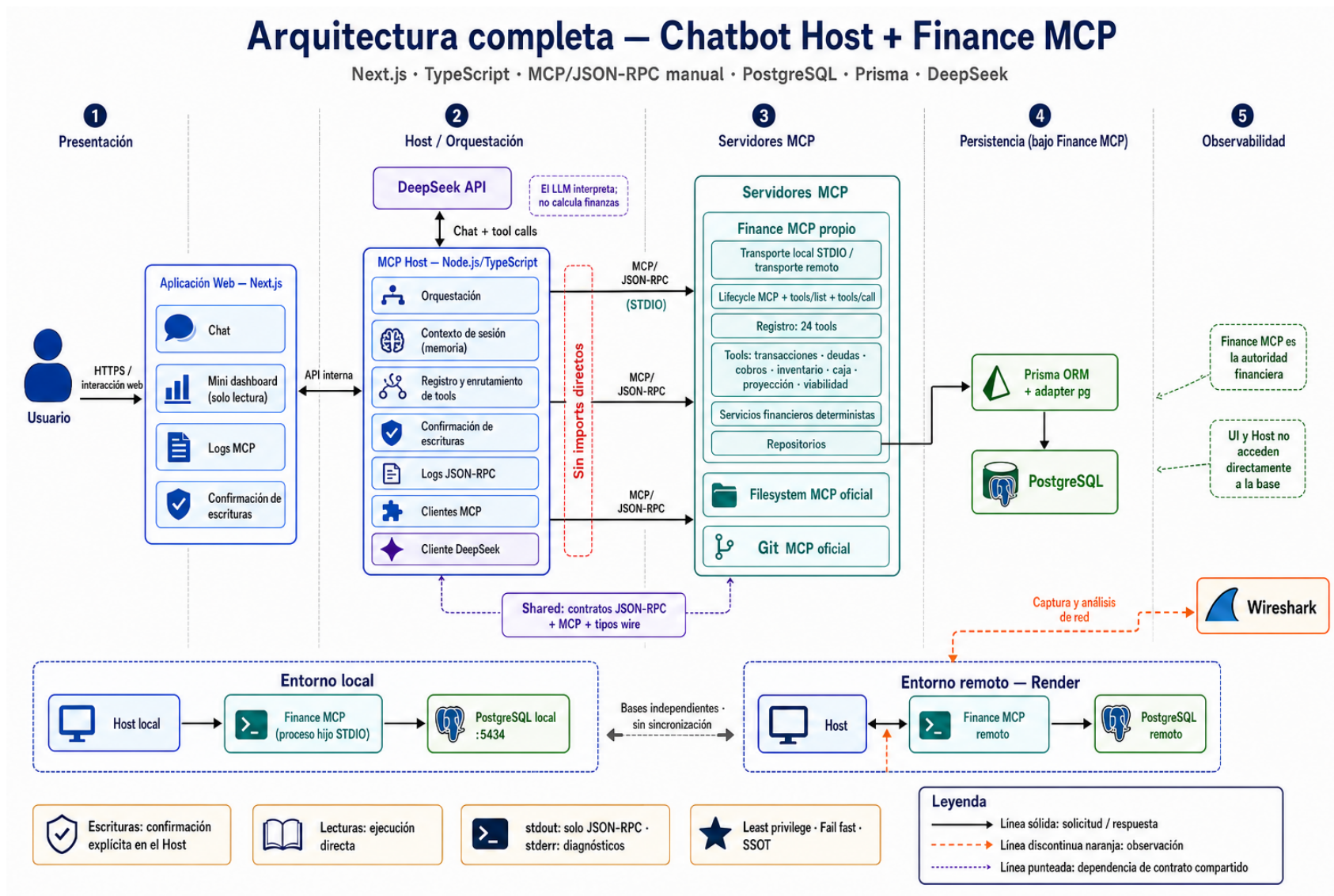
La arquitectura separa la interfaz, la coordinación, los servidores MCP y la persistencia. El navegador se comunica únicamente con Route Handlers de Next.js. El Host crea clientes MCP, descubre sus catálogos y enruta cada tool al cliente propietario. El cliente DeepSeek interpreta mensajes y solicita tools cuando se requiere información externa. No obtiene acceso directo a Prisma, PostgreSQL ni a transportes MCP.

Finance MCP es la autoridad para reglas financieras y persistencia. Prisma comunica el servidor con PostgreSQL. Las respuestas financieras estructuradas se devuelven al Host sin exponer modelos internos ni detalles de conexión. El Host aplica confirmaciones a todas las escrituras de Finance, Filesystem y Git. El navegador envía una decisión de confirmación, nunca reenvía argumentos de la tool.

La composición Web es progresiva. El dashboard inicia solamente Finance MCP. La apertura del chat extiende el runtime con DeepSeek, Filesystem MCP y Git MCP. Los logs MCP se conservan en memoria del proceso y se agrupan por lifecycle, dashboard y conversación actual. Esta separación evita que una consulta de dashboard dependa de la configuración del modelo o de servidores locales no financieros.

La selección de Finance MCP conserva el mismo identificador lógico y catálogo público. El modo local utiliza un proceso hijo y STDIO. El modo remoto utiliza un cliente Streamable HTTP hacia el endpoint público. La base de datos de cada modo es independiente.

Figura 1. Arquitectura completa del proyecto



Fuente: documentación de arquitectura del repositorio.

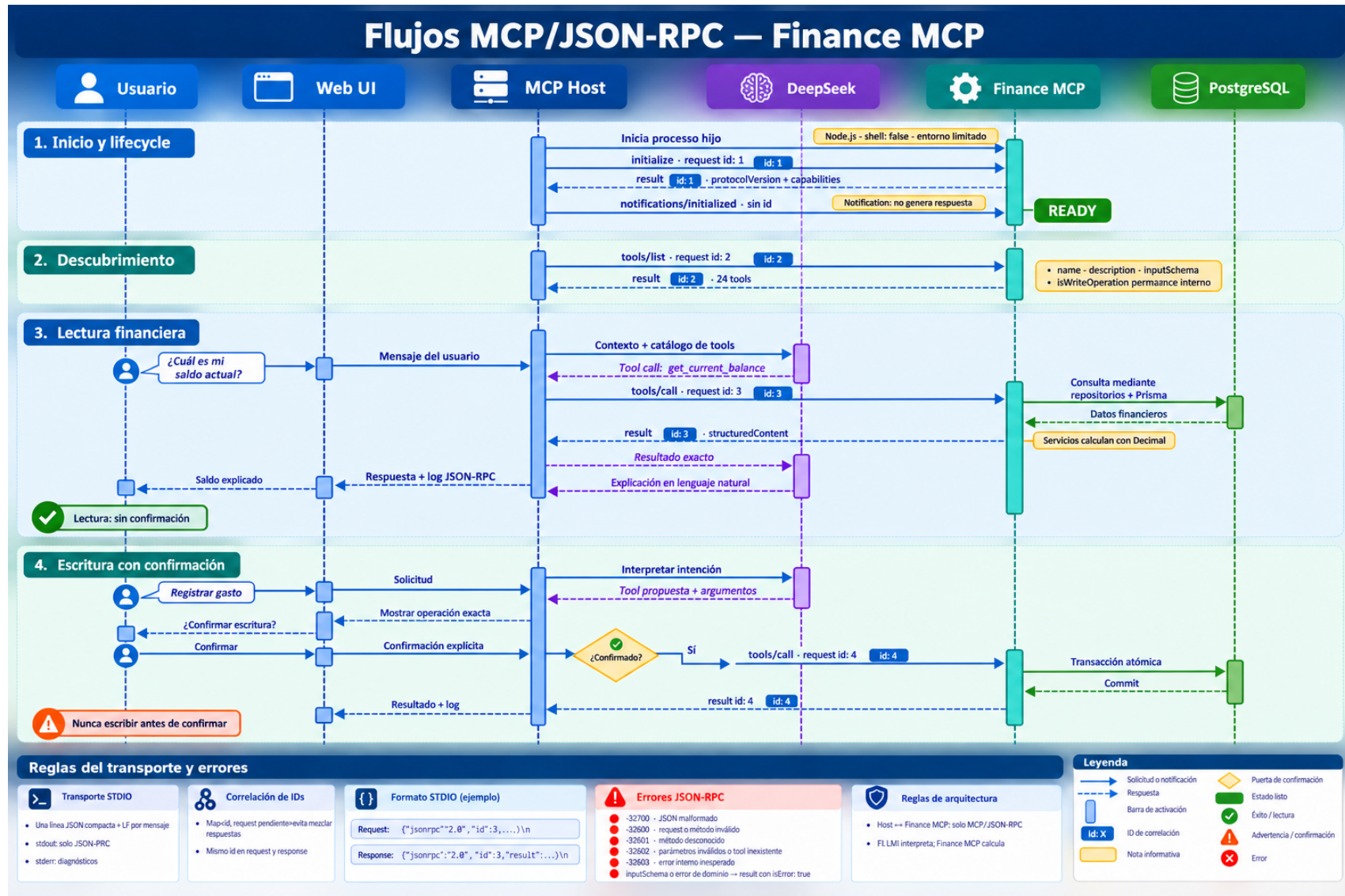
Implementación manual de MCP y JSON-RPC

MCP utiliza JSON-RPC 2.0 como capa de intercambio de datos. Cada request contiene la versión JSON-RPC, un método, parámetros cuando corresponden y un identificador para correlacionar la respuesta. Una notification carece de identificador y no recibe respuesta JSON-RPC. Esta semántica permite distinguir requests, responses y notifications dentro de cualquier transporte compatible [2], [4].

El lifecycle implementado inicia con `initialize`. El cliente anuncia la versión MCP, sus capacidades y su identidad. Finance MCP responde con su versión, capacidades y nombre. El cliente envía después `notifications/initialized`. Luego obtiene las definiciones mediante `tools/list` y puede invocar una operación mediante `tools/call`. El servidor rechaza `discovery` y `tool calls` antes del estado `READY`.

La implementación no usa un SDK MCP. El servidor valida envelopes JSON-RPC, conserva estado de lifecycle, genera respuestas correlacionadas, aplica schemas de entrada y traduce errores esperados a resultados MCP seguros. Los errores de envelope usan códigos JSON-RPC. Los errores de validación de una tool se representan en un resultado MCP con `isError`.

Figura 2. Flujos JSON-RPC y confirmación de escritura



Fuente: documentación de arquitectura del repositorio.

Especificación de Finance MCP

Finance MCP administra transacciones, deudas, cuentas por cobrar, inventario, saldo, flujo de caja, proyección y viabilidad de compra. El catálogo productivo contiene veinticuatro tools. Los montos viajan como strings decimales y se procesan con aritmética decimal. Las fechas usan el formato YYYY-MM-DD. La moneda del dominio es GTQ. Las respuestas exitosas incluyen texto determinista y datos canónicos en structuredContent. Los fallos esperados se señalan con isError sin exponer SQL, Prisma o secretos.

Tool	Operación	Parámetros principales	Resultado principal
record_income	Escritura	cuenta, categoría, monto, fecha	Transacción creada
record_expense	Escritura	cuenta, categoría, monto, fecha	Transacción creada
list_transactions	Lectura	rango, tipo, cuenta, categoría	Lista de transacciones
update_transaction	Escritura	identificador y cambios	Transacción actualizada
delete_transaction	Escritura	identificador	Transacción eliminada
record_debt	Escritura	descripción, monto, vencimiento, prioridad	Deuda creada
list_debts	Lectura	estado, prioridad, fecha límite	Lista de deudas
update_debt	Escritura	identificador y cambios	Deuda actualizada
mark_debt_paid	Escritura	identificador	Deuda pagada
delete_debt	Escritura	identificador	Deuda eliminada
record_receivable	Escritura	descripción, monto, fecha esperada, confianza	Cuenta por cobrar creada
list_receivables	Lectura	estado, confianza, fecha límite	Lista de cuentas por cobrar
update_receivable	Escritura	identificador y cambios	Cuenta por cobrar actualizada
mark_receivable_collected	Escritura	identificador	Cuenta por cobrar cobrada
delete_receivable	Escritura	identificador	Cuenta por cobrar eliminada
create_product	Escritura	nombre, stock, costos, mínimo	Producto creado
list_products	Lectura	sin parámetros	Lista de productos
update_product	Escritura	identificador y cambios	Producto actualizado
record_inventory_movement	Escritura	producto, tipo, cantidad, fecha	Movimiento creado
list_low_stock_products	Lectura	sin parámetros	Productos con stock bajo
get_current_balance	Lectura	sin parámetros	Saldo y cuentas
get_cash_flow_summary	Lectura	fechas inicial y final	Ingresos y egresos
project_cash_flow	Lectura	horizonte en días	Escenarios de proyección
evaluate_purchase_viability	Lectura	compra y horizonte	Evaluación de viabilidad

La especificación completa, los schemas y las reglas de error se conservan en docs/finance-mcp-tools.md. La tabla anterior funciona como síntesis del contrato, no como sustituto de esa fuente técnica.

Transportes, endpoints y configuración

El modo local utiliza STDIO. Cada mensaje JSON-RPC compacto se intercambia por la entrada y salida estándar del proceso Finance MCP. Este modo elimina tráfico de red y se utiliza junto con PostgreSQL local. El Host conserva un cliente STDIO y registra el transporte como STDIO.

El modo remoto utiliza Streamable HTTP. El endpoint público es <https://finanzas-mcp-server.onrender.com/mcp>. La versión MCP es 2025-11-25. POST /mcp transporta initialize, notifications/initialized, tools/list y tools/call. DELETE /mcp cierra la sesión remota. El servidor responde con 200 para requests exitosas, 202 para la aceptación de la notification y 204 para el cierre. La implementación anuncia JSON y event stream en Accept, pero no usa SSE porque el servidor no inicia mensajes hacia el cliente.

Después de initialize, las requests remotas incluyen los nombres de header MCP-Session-Id y MCP-Protocol-Version. Sus valores no se exponen en código versionado, logs Web ni documentación. La configuración selecciona el modo mediante FINANCE_MCP_MODE. El endpoint remoto se valida como HTTPS con la ruta exacta /mcp. Un fallo remoto no activa un fallback local.

Render aloja Finance MCP y PostgreSQL remoto. La inicialización protegida de la base aplica migraciones existentes con prisma migrate deploy y usa un seed canónico únicamente en una base vacía [6], [7]. Filesystem MCP y Git MCP no se despliegan de forma remota.

Ejemplos de uso

Una lectura de saldo se ejecuta inmediatamente después del lifecycle. La solicitud usa tools/call con el nombre get_current_balance y un objeto de argumentos vacío. La respuesta contiene structuredContent con la moneda GTQ y un saldo decimal. La interfaz presenta el resultado sin recalcularlo.

```
tools/call
name get_current_balance
arguments objeto vacío
resultado structuredContent con moneda GTQ y saldo decimal
```

Una escritura se detiene en el Host. Una solicitud record_income requiere cuenta, categoría, monto decimal, fecha y una descripción opcional. El Host conserva la operación exacta y presenta una tarjeta de confirmación con los argumentos. La ejecución ocurre solamente después de una decisión explícita. La cancelación descarta la operación y no invoca la tool.

```
tools/call
name record_income
arguments cuenta, categoría, monto decimal, fecha y descripción
resultado pendiente de confirmación del Host
```

Interfaz Web y controles HCI

La interfaz Web reúne tres vistas. El dashboard financiero presenta consultas de solo lectura. El chat mantiene contexto dentro de una pestaña. El panel Logs MCP presenta payloads sanitizados de lifecycle, dashboard y conversación actual. Las tres vistas usan la misma instancia financiera cuando el proceso ya la inicializó.

Las respuestas completadas del modelo aceptan CommonMark y GFM mediante un renderer seguro. Los mensajes de usuario, errores y controles del Host permanecen como texto plano. HTML crudo, imágenes remotas y protocolos de enlace peligrosos no se interpretan. Las confirmaciones de escritura se muestran como tarjetas inline con descripción, servidor, tool y argumentos colapsables.

La navegación usa tabs accesibles, foco visible y controles con etiquetas. Los estados de carga, actualización y error permanecen visibles. El sistema visual utiliza contraste, bordes definidos y jerarquía tipográfica para separar dashboard, conversación, logs y confirmaciones. La interfaz no accede directamente a DeepSeek, MCP, Prisma ni PostgreSQL.

Verificación y calidad

La calidad se validó mediante pruebas unitarias, integraciones de Finance MCP con PostgreSQL efímero, integración HTTP, pruebas de Git MCP y un escenario local completo. La matriz final confirmó lifecycle, discovery, operaciones de lectura, validación de errores, confirmaciones y límites de sandbox. El escenario Finance a Filesystem a Git requiere tres confirmaciones independientes.

La regresión final automatizada completó las suites de comportamiento general, Finance STUDIO, Finance HTTP, Git, demo local, tipos, lint y build. La validación remota confirmó el catálogo de veinticuatro tools, lecturas, proyecciones, viabilidad y una mutación reversible restaurada por MCP. El estado final registrado corresponde a 269 pruebas aprobadas. Los checks remotos se mantienen separados de la suite general porque dependen de infraestructura desplegada [1].

Clasificación JSON-RPC y MCP

La captura final seleccionada contiene una sesión de solo lectura entre el Host productivo y Finance MCP remoto. Antes de aplicar el key log local, el contenido de aplicación aparece como TLS Application Data cifrada. Después del descifrado local, los métodos, IDs y status HTTP se correlacionaron con el resumen seguro del Host. No se versionaron la captura, claves, headers completos, payloads completos ni identificadores de sesión.

Frames	Stream	Método e interacción	ID	Clasificación	HTTP
13 / 15	0	POST /mcp initialize	1	Lifecycle request y response	200
16 / 18	0	POST /mcp notifications/initialized	-	Lifecycle notification	202
31 / 35	1	POST /mcp tools/list	2	Discovery request y response	200
37 / 39	0	POST /mcp tools/call get_current_balance	3	Tool request y response	200
40 / 42	0	DELETE /mcp	-	Gestión de transporte	204

notifications/initialized no contiene ID. Su 202 expresa aceptación HTTP y no una respuesta JSON-RPC. DELETE /mcp tampoco es una solicitud JSON-RPC. Los IDs 1, 2 y 3 correlacionan requests y responses. tools/list descubrió veinticuatro tools. La única ejecución fue get_current_balance, clasificada como lectura. No aparece ninguna tool de escritura ni un mensaje de error en la evidencia [8].

Análisis por capas

Enlace

La captura se realizó desde la interfaz Wi-Fi seleccionada durante la ejecución. Npcap expuso el tráfico como Ethernet II con encapsulación ether. Se observaron 48 tramas y el primer frame cliente a gateway transportó IPv4. Las direcciones MAC se omiten por privacidad. La evidencia no contiene encabezados IEEE 802.11 ni permite concluir comportamiento de radio.

Red

Los extremos IPv4 observados fueron 192.168.1.38 y 216.24.57.15. El Host originó tráfico con TTL 128. Las respuestas remotas mostraron TTL 54 o 57 según el stream. El filtro de captura limitó el conjunto al puerto TCP 443 y a la dirección resuelta antes de la captura. Por ello no existe tráfico DNS dentro del archivo. El SNI público asocia TLS con el endpoint, sin constituir evidencia DNS. NAT y routers intermedios son inferencias, no observaciones directas.

Transporte y TLS

Se observaron dos conexiones TCP. El stream 0 usó el puerto efímero 50811 y transportó initialize, la notification, la lectura y el cierre. El stream 1 usó 50812 y transportó discovery. Ambas conexiones terminaron en 443. Los handshakes TCP ocurrieron en los frames 1 a 3 y 19 a 21. Los FIN del cliente aparecieron en 43 y 44. Los FIN del servidor aparecieron en 45 y 47.

Cada stream negoció TLS 1.3 con TLS_AES_256_GCM_SHA384. El Client Hello ocurrió en los frames 5 y 23. El Server Hello ocurrió en 9 y 27. ALPN negoció http/1.1. El valor legacy 0x0303 no representa la versión final. La versión negociada fue 0x0304, TLS 1.3.

Aplicación

La pila observada es MCP 2025-11-25 sobre JSON-RPC 2.0, Streamable HTTP, HTTP/1.1, TLS 1.3, TCP, IPv4 y Ethernet expuesto por Npcap. PostgreSQL existe como componente arquitectónico detrás de Finance MCP, pero no fue un par de red de esta captura. DeepSeek, SSE, HTTP/2 y escrituras tampoco participaron en la sesión seleccionada [9].

Dificultades, soluciones y aprendizajes

Dificultad	Solución aplicada	Aprendizaje técnico
Implementación manual de MCP	Separación entre handler, lifecycle y transporte	El contrato JSON-RPC puede reutilizarse entre STDIO y HTTP
Orden del lifecycle	Estado READY validado antes de discovery y tool call	La secuencia de inicialización condiciona la interoperabilidad
Exactitud monetaria	Montos como strings y aritmética decimal en Finance MCP	JavaScript numérico no debe representar dinero de dominio
Escrituras solicitadas por LLM	Operación retenida por el Host y confirmación explícita	El modelo puede proponer acciones sin controlar su ejecución
Transporte remoto	Cliente Streamable HTTP con sesión interna y timeout	Un contrato estable permite equivalencia local y remota
Captura TLS	Key log local temporal y correlación con logs seguros	El cifrado protege payloads y aún permite análisis autorizado

Conclusiones y comentario del proyecto

1. MCP permite exponer herramientas mediante un contrato independiente del modelo y del transporte seleccionado.
2. La separación entre interfaz, Host, clientes MCP, servidores y persistencia reduce acoplamiento y preserva la autoridad del dominio financiero.
3. STDIO y Streamable HTTP mantuvieron el mismo catálogo público de Finance MCP. La diferencia se concentró en el transporte y la ubicación de la persistencia.
4. La confirmación explícita conservada por el Host evita que una petición de escritura generada por el modelo se ejecute sin intervención del usuario.
5. La captura final demostró lifecycle, discovery, una lectura y cierre de sesión sobre TCP, TLS y HTTP. El descifrado local permitió clasificar JSON-RPC sin publicar material sensible.

El alcance integra protocolos, servicios, persistencia, interfaz y evidencia de red dentro de una implementación verificable. La combinación de límites explícitos, pruebas reproducibles y documentación técnica permite evaluar el comportamiento sin presentar datos ficticios como evidencia ni ampliar el MVP con capacidades ajenas al objetivo del curso.

Referencias

- [1] Universidad del Valle de Guatemala, "Proyecto 1 - Uso de un protocolo existente," guía del curso CC3067 Redes, 2026.
- [2] JSON-RPC Working Group, "JSON-RPC 2.0 Specification." Disponible en <https://www.jsonrpc.org/specification>. Consulta 1 de septiembre de 2026.
- [3] Model Context Protocol, "Architecture overview." Disponible en <https://modelcontextprotocol.io/docs/learn/architecture>. Consulta 1 de septiembre de 2026.
- [4] Model Context Protocol, "Transports, protocol revision 2025-11-25." Disponible en <https://modelcontextprotocol.io/specification/2025-11-25/basic/transports>. Consulta 1 de septiembre de 2026.
- [5] Model Context Protocol, "Model Context Protocol Servers." Disponible en <https://github.com/modelcontextprotocol/servers>. Consulta 1 de septiembre de 2026.

-
- [6] Render, "Create and connect to Render Postgres." Disponible en <https://render.com/docs/postgresql-creating-connecting>. Consulta 1 de septiembre de 2026.
- [7] Prisma, "prisma migrate deploy." Disponible en <https://docs.prisma.io/docs/cli/migrate/deploy>. Consulta 1 de septiembre de 2026.
- [8] Proyecto Finance MCP, "Final MCP message classification." Disponible en <docs/wireshark/final-mcp-message-classification.md>. Consulta 1 de septiembre de 2026.
- [9] Proyecto Finance MCP, "Final remote communication layer analysis." Disponible en <docs/wireshark/final-remote-layer-analysis.md>. Consulta 1 de septiembre de 2026.